

FY-26: Career Preview Program Project Review (CPP-3)

Vellore Institute of Technology, Vellore

Project Overview

Making encrypted QUIC traffic visible, classifiable, and controllable - without breaking privacy

Details

Project Name: QUIC Traffic Identification, Classification & Policing

College Vellore Institute of Technology, Vellore

Duration February 2026 - July 2026

HPE Mentor Sivakumar Ramasamy

VIT Mentor Dr. Ruby D

Project Participants

- Kavya Jain
- Suvidh Jain
- Shubham Kumar
- Vansh Chani
- Rajan Hasija

Source Code: https://github.com/SuvidhJ/QUIC_Identification_and_Policing_HPE_CPP

Problem Statement

The widespread adoption of the QUIC protocol has made traffic classification and Quality-of-Service enforcement significantly harder as it encrypts most transport-level metadata.



Problem Overview

The rapid adoption of QUIC for HTTP/3 improves connection latency and reliability. However, its UDP-based architecture and encrypted transport-layer metadata make traffic identification significantly more challenging than traditional TCP-based protocols.



Need for QUIC Classification

Accurate classification of QUIC and Non-QUIC traffic is essential for effective network monitoring and management. As multiple applications with diverse QoS requirements use QUIC, protocol detection alone is insufficient, making application-level traffic classification equally important.



Our Objective

Design a system to detect QUIC traffic from raw network packets, classify it into application categories (Video Streaming, Web Browsing, File Download) using observable flow features, and enforce dynamic QoS policies without decrypting packet payloads.

Problem Statement



THE SHIFT

QUIC now carries a large and rapidly growing share of internet traffic. Google reports ~30% of its global egress runs on QUIC, and HTTP/3 - built on QUIC - is used by over 30% of the top 10 million websites (YouTube, Meta, Cloudflare).

Once niche, now mainstream.



THE TECHNICAL PROBLEM

QUIC runs over UDP, encrypts almost all transport-layer metadata (including much of the handshake), and multiplexes streams within a single connection.

Classical Deep Packet Inspection (DPI) - the technique enterprise networks relied on for two decades - becomes

blind to it.



WHY THIS MATTERS FOR HPE & CUSTOMERS

- **QoS enforcement breaks**
can't prioritise Zoom over background downloads
- **Security blind-spot**
malware & exfiltration hide inside QUIC
- **Capacity planning becomes guesswork**
no visibility into video vs. web vs. bulk
- **Existing DPI is heavy & privacy-hostile**
costly, adds latency, often breaks TLS



OUR OBJECTIVE

Design a system to:

- **Detect QUIC traffic** from raw network packets
- **Classify** into Video / Web / File Download via flow features
- **Enforce dynamic QoS** — without decryption, privacy-preserving

Solution Proposal

We developed an end-to-end, four-stage pipeline to capture, identify, classify, and police encrypted network traffic without relying on standard Deep Packet Inspection (DPI).

MODULE 1: RAW PACKET EXTRACTION

Bypassing standard libraries to manually decode packets at the NIC level (ETH_P_ALL) for real-time, zero-latency performance.

MODULE 2: QUIC IDENTIFICATION

Extracting heuristic evidence (headers) and normalizing 5-tuples to robustly detect QUIC flows via a multi-tier framework.

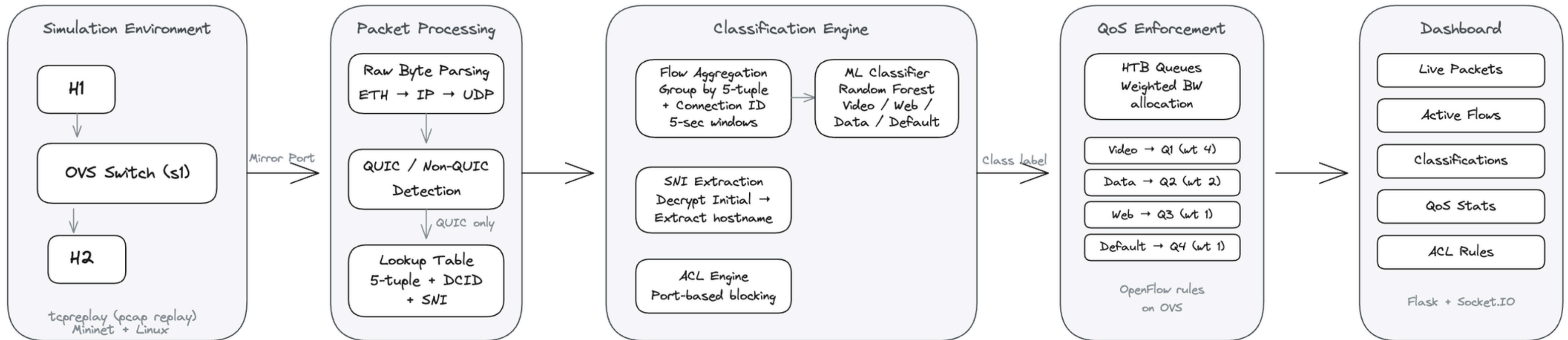
MODULE 3: QUIC CLASSIFICATION

Training and deploying a SMOTE-balanced XGBoost machine learning model on 14 statistical flow features to categorize traffic.

MODULE 4: QOS POLICING

Applying autonomous, dynamic bandwidth allocation on Open vSwitch (OVS) based on real-time ML traffic inferences.

Overall Flow



Raw Byte Parsing

Instead of relying on heavy third-party libraries (Scapy, aioquic), we manually decode Ethernet, IP, and UDP layers using Python's socket and struct modules.

Capturing the Packet

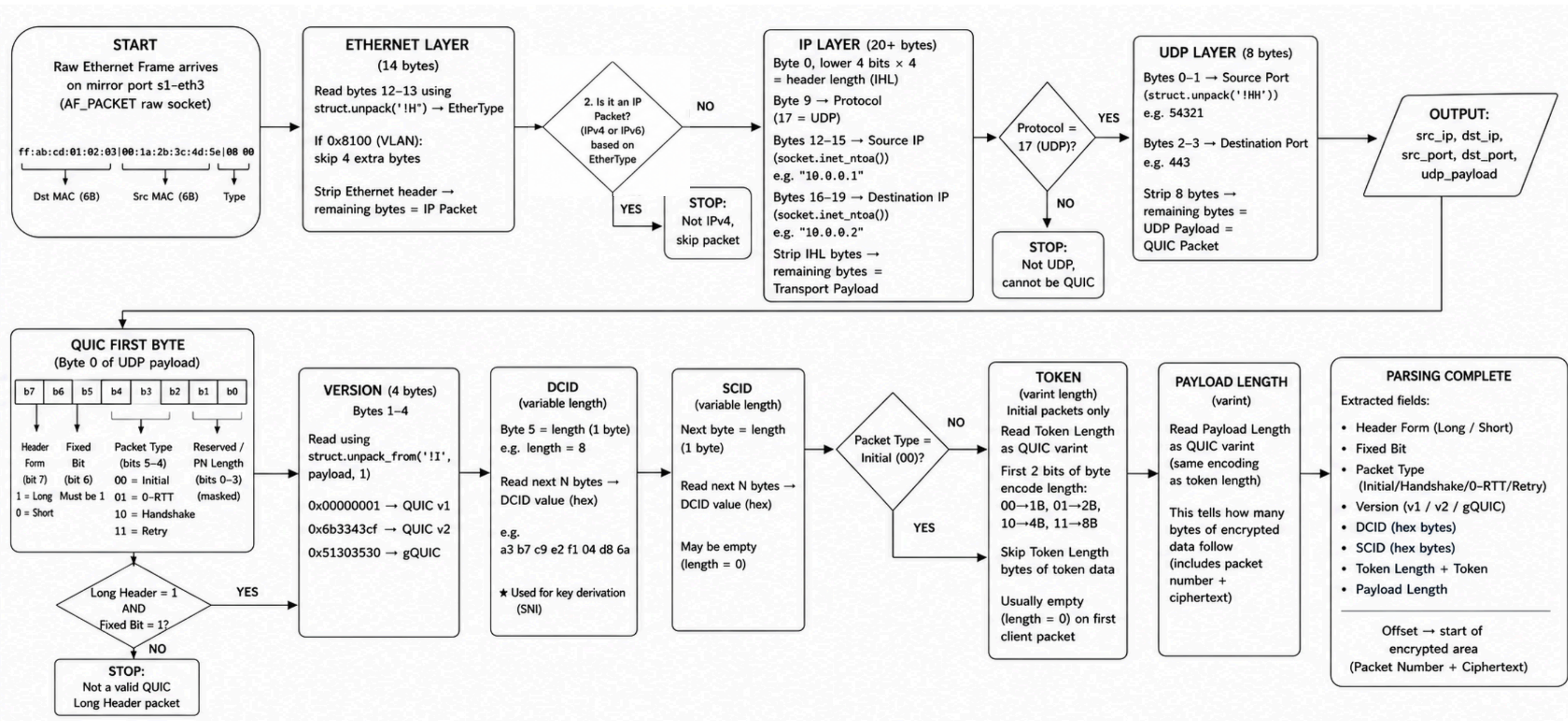
We open a raw socket (AF_PACKET) bound to our mirror port (s1-eth3). This gives us every frame that passes through the switch — as raw bytes, straight from the network card. No filtering, no preprocessing. We get the full Ethernet frame exactly as it was sent.

Peeling the Layers

Each packet is wrapped in layers. First we strip the Ethernet header (14 bytes) to get the IP packet. Then we read the IP header to find source/destination IPs and confirm it's UDP. Then we strip the UDP header (8 bytes) to get the payload. What's left is the QUIC data — the part we actually need.

Reading the QUIC Header

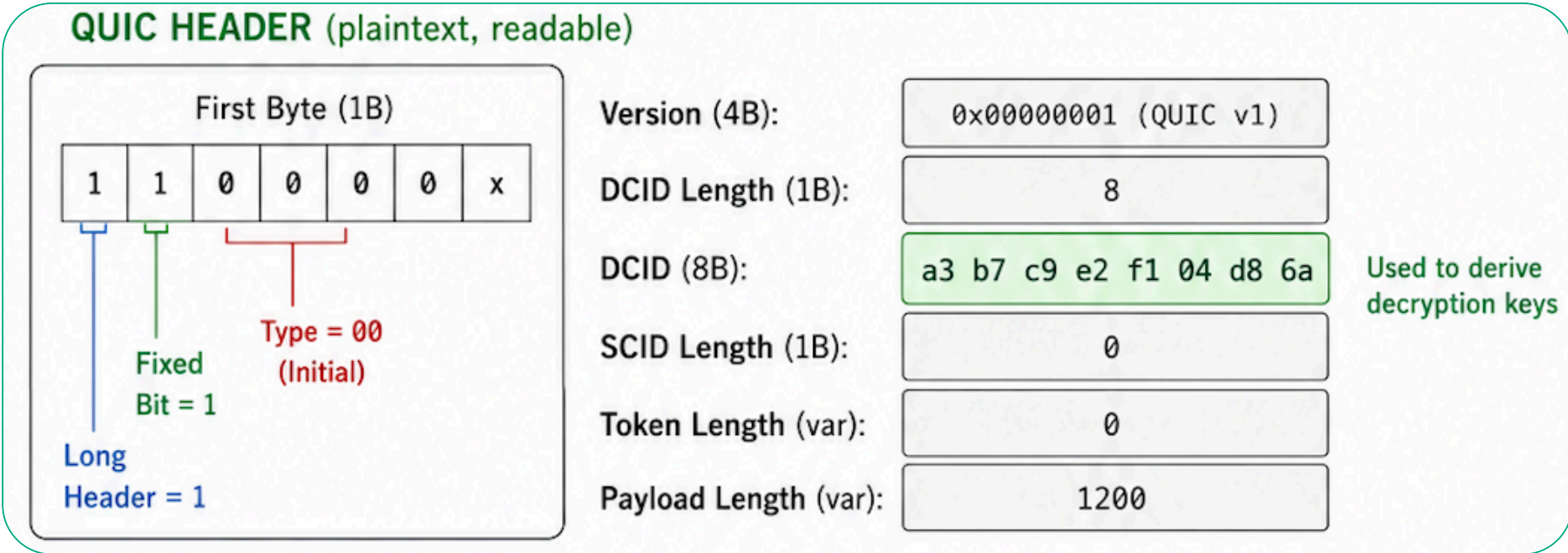
From the QUIC payload, we read the first byte to determine if it's a Long or Short header. For Long Headers, we extract the version (4 bytes), then the Connection IDs (DCID and SCID) by reading their length bytes first, then the actual IDs. We also identify the packet type — Initial, Handshake, 0-RTT, or Retry — from bits 4-5 of the first byte.



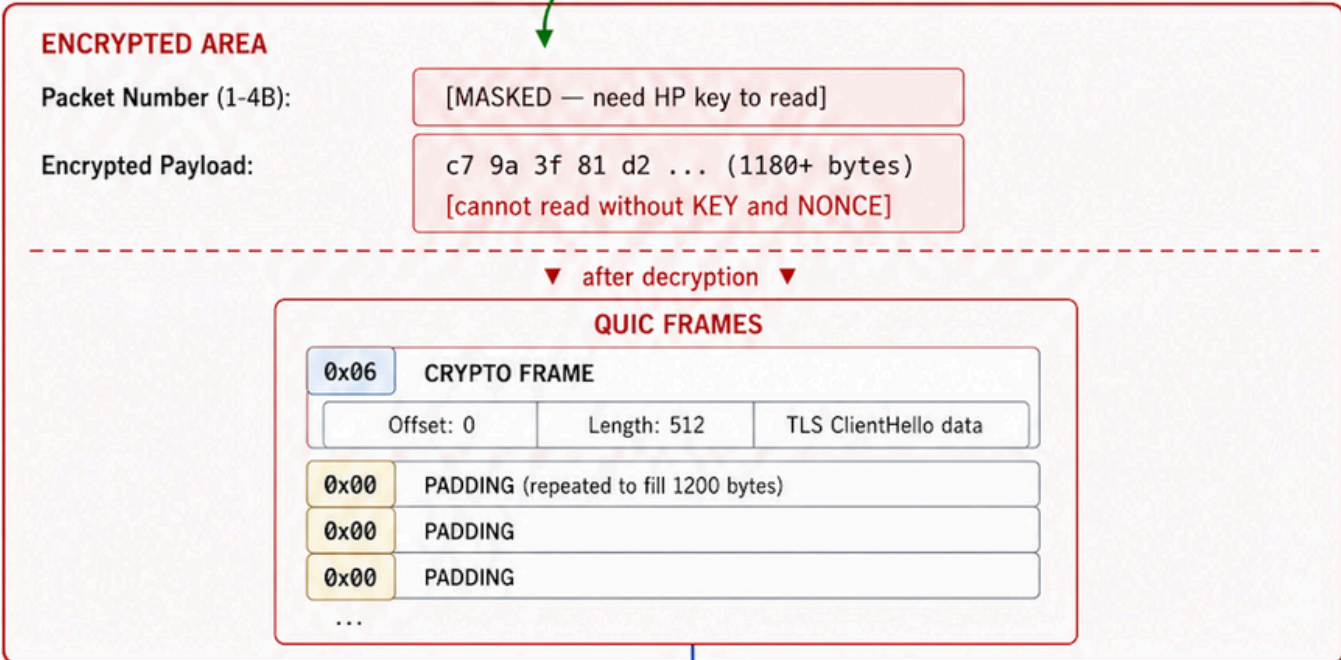
Manual Decoding Pipeline

- 1 Raw Socket Capture:** Captures modified frames using ETH_P_ALL directly from eth0. No OS decoding.
- 2 L2/L3 Decoding:** Manually extracts MACs and EtherType; validates IPv4 and parses IHL for header length. TCP is instantly dropped.
- 3 QUIC Heuristics:** Inspects bits in the UDP payload for Long/Short headers, the Fixed bit (0x40), and extracts Version fields.
- 4 Flow Normalization:** Builds a 5-tuple (sorting IPs/Ports consistently) to track bidirectional flows under a single identity.

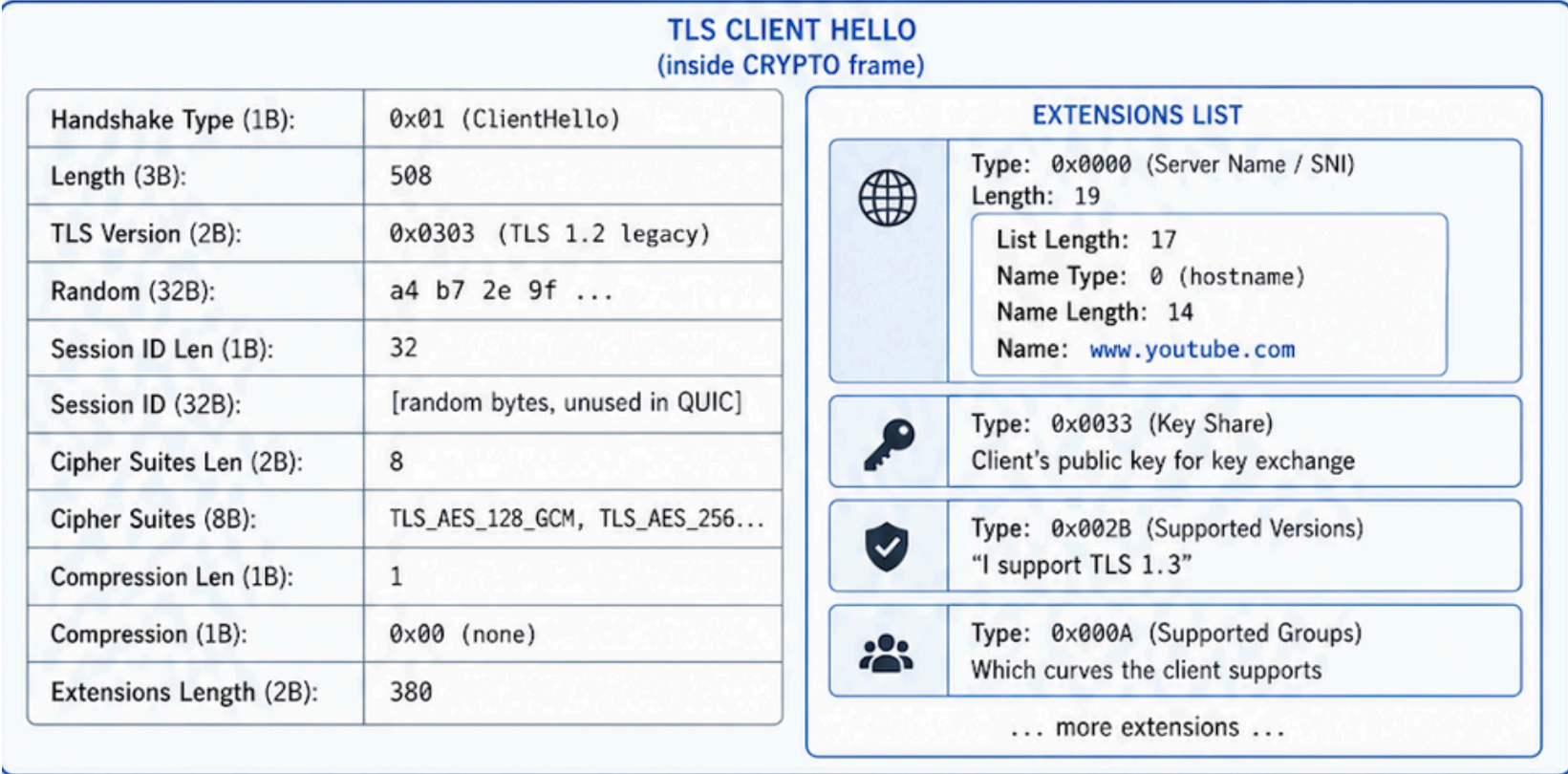
Extracting The SNI From Intial Packet



(a) QUIC Headers



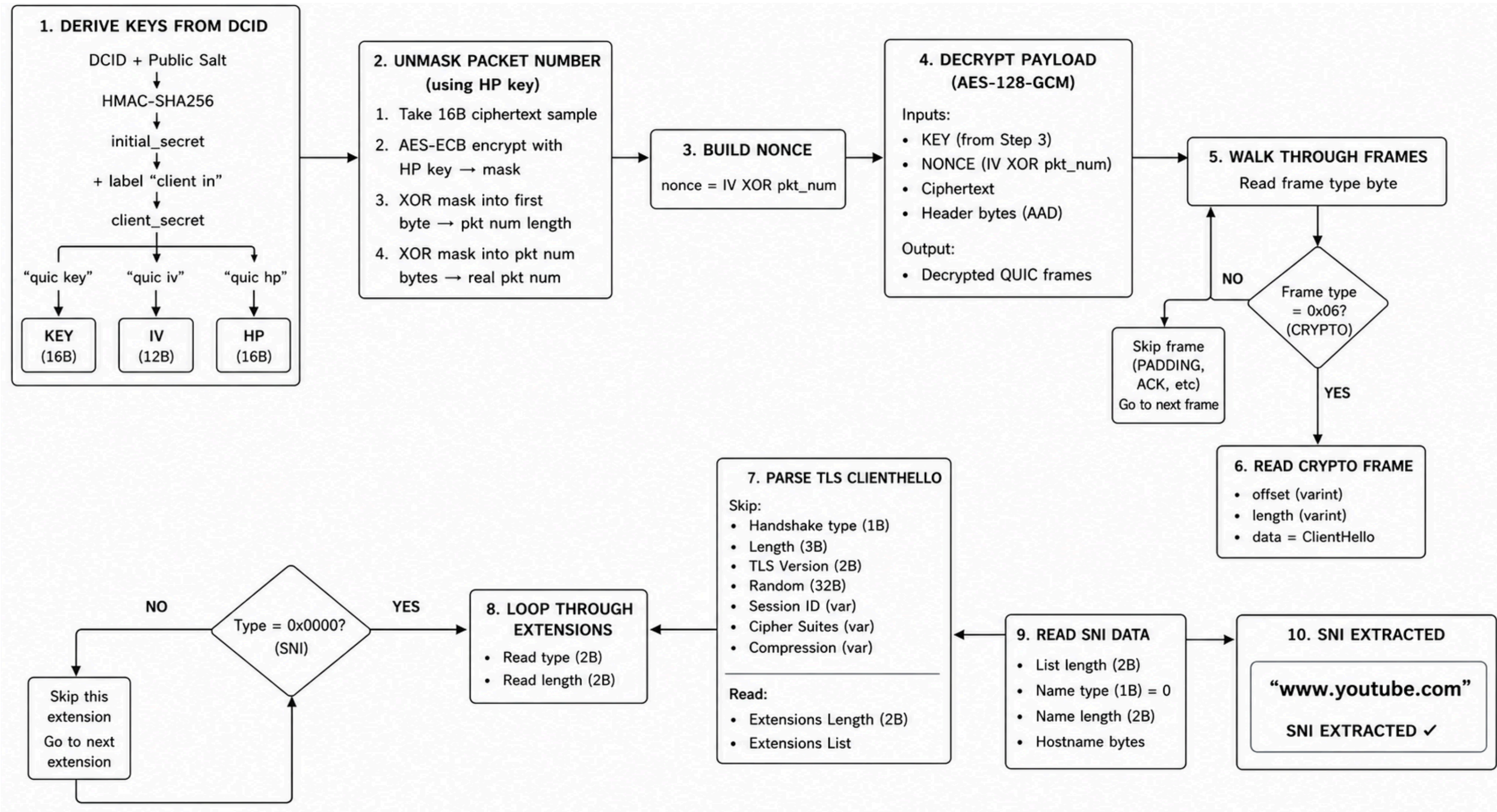
(b) Encrypted Payload



(c) Decrypted client hello that carry the sni

The Initial ClientHello packet is decrypted using a key derived entirely from the Destination Connection ID (DCID) and a fixed public salt defined in RFC 9001 — no secrets or handshake completion required.

Generating the Key to decrypt initial packet



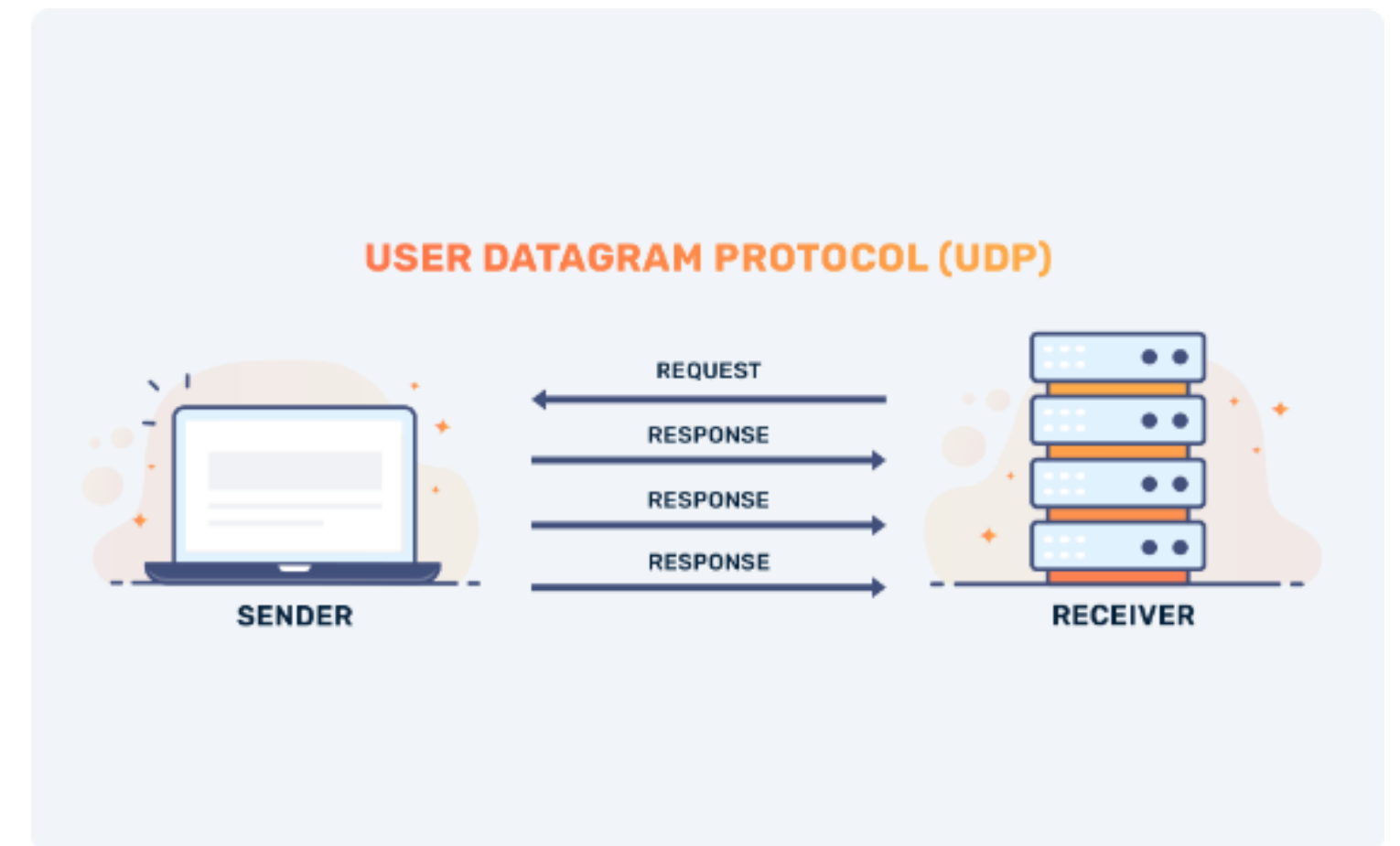
QUIC Header Interpretation

Bit-Level Inspection

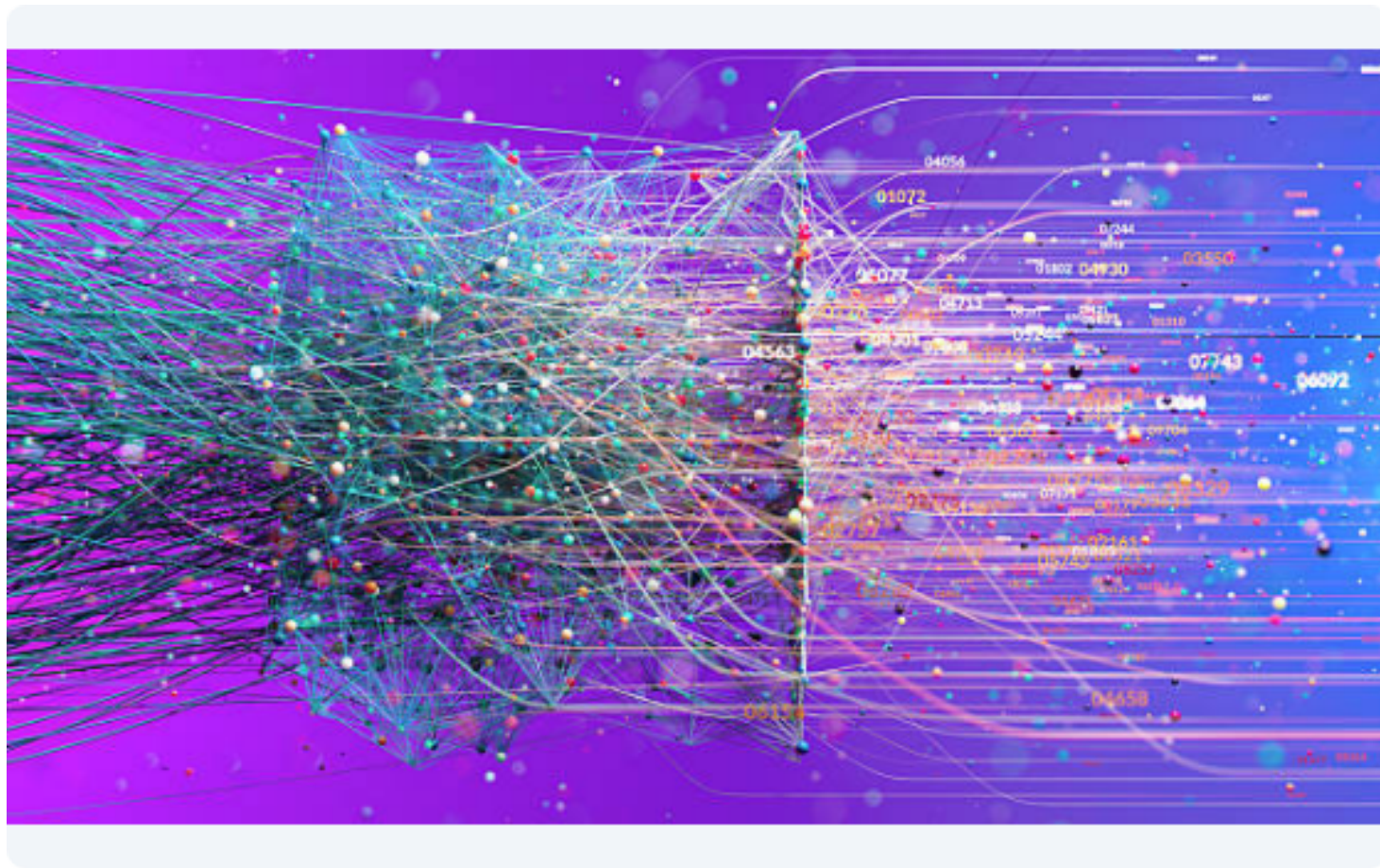
- **Long Header Bit:** Most significant bit. Signals handshake establishment, RTT, Version Negotiation, and Retry packets.
- **Fixed Bit (0x40):** Required by RFC to be 1. Used to distinguish QUIC from other UDP noise.

Heuristic Evidence

If the Long Header is set, we extract the Version Field (4-byte ID) and compare against known variants.



Flow Normalization & Evidence



The Normalized 5-Tuple

To ensure bidirectional traffic (Client↔Server) is recognized as one flow, we sort IP/Port pairs into a consistent order before hashing

Evidence Buffering

Feature extraction begins after the first 5 packets of a normalized bidirectional flow are collected. We aggregate statistics across the window:

- Proportion of packets with Long Header set.
- Diversity of distinct QUIC versions.
- Frequency of specific packet types (Initial, Handshake, 0-RTT).

QUIC Identification

Hybrid Classifier: First, incoming packets are grouped into a 5-packet flow to extract protocol-specific and statistical features. The flow is then processed through a three-tier framework.



Tier 1: Deterministic

Performs rapid deterministic detection utilizing explicit QUIC protocol signatures.

Checks include transport protocol matching, the fixed bit, long header presence, and valid version information verification.



Tier 2: Handshake

If Tier 1 is unconfident, the flow proceeds to deeper packet inspection.

The system actively checks for specific QUIC handshake packet types (Initial, Handshake, Retry, or 0-RTT).



Tier 3: ML & Lookup

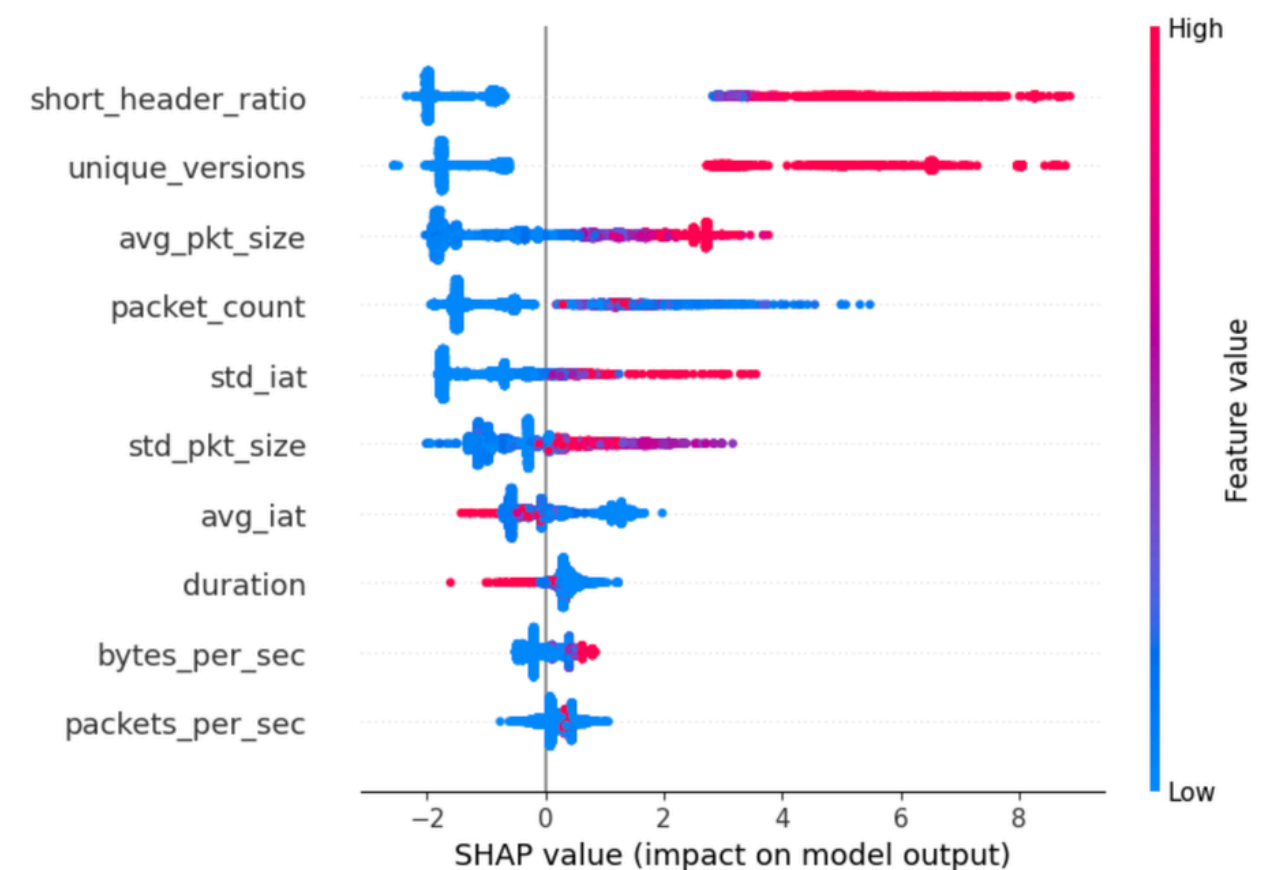
Tier 3a - cached-flow lookup: First performs a memory lookup using previously identified Connection 5-tuples.

Tier 3b - ML fallback: If no rule-based match is found, an ML model classifies the flow using statistical flow features, with key contributors including short header ratio, average packet size, packet count, bytes per second etc. producing one of three outputs: QUIC, QUIC_PROBABLE, or NON-QUIC.

Tier 3b - ML Fallback

- Original dataset contained an imbalanced distribution of QUIC and Non-QUIC traffic.
- The majority class (Non-QUIC) was randomly downsampled to eliminate model bias.
- Final balanced dataset: 11,761 QUIC flows and 11,761 Non-QUIC flows
- Model Performance (XGBoost)
 - Accuracy: 99.83%
 - Precision: 99.75%
 - Recall: 99.91%
 - F1-Score: 99.83%

Feature	Importance
Short Header Ratio	35.67%
Unique QUIC Versions	27.97%
Average Packet Size	17.15%
Packet Count	8.83%
Bytes/sec	3.51%



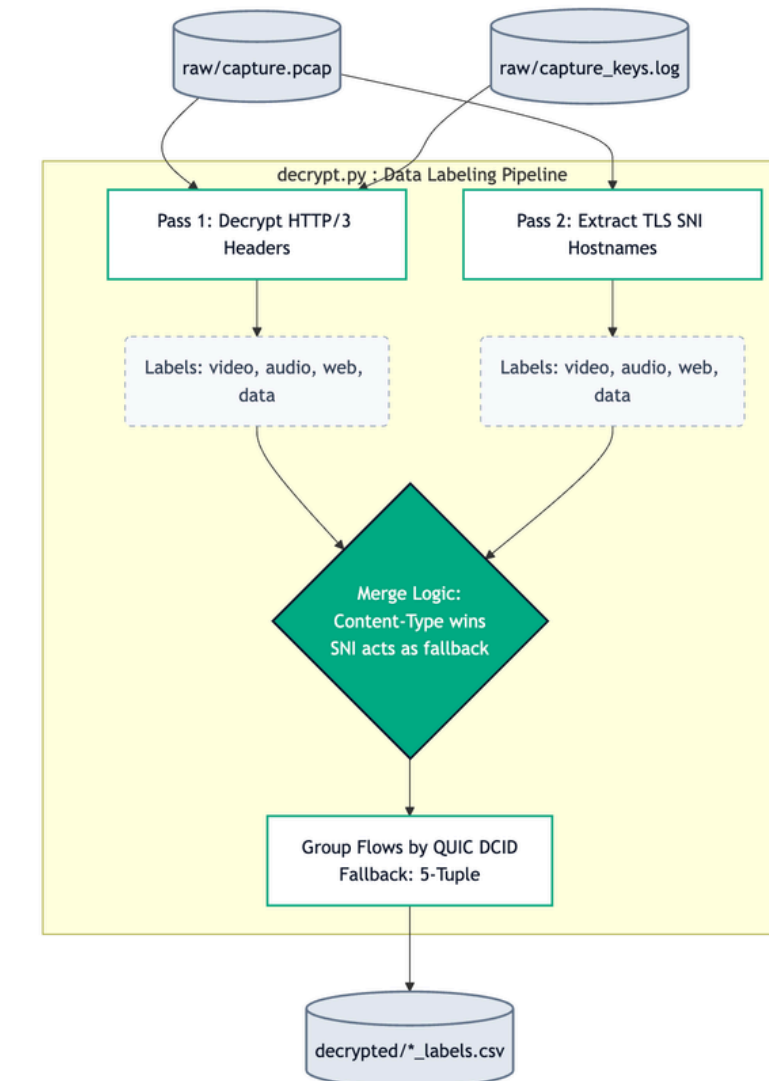
ML Model: Establishing Ground Truth

To train the ML classifier, we built a secure labeling pipeline to capture real-world traffic classes.

1. Traffic Capture: Used tshark alongside Chrome TLS key logs to capture raw UDP traffic.

2. Decryption & Labeling: Decrypted HTTP/3 headers. Flows were assigned labels (video, audio, web, data) based primarily on content-type, with TLS SNI hostnames as a fallback.

3. Flow Keying: Grouped flows utilizing the QUIC Destination Connection ID (DCID) as the primary key to survive IP migrations.

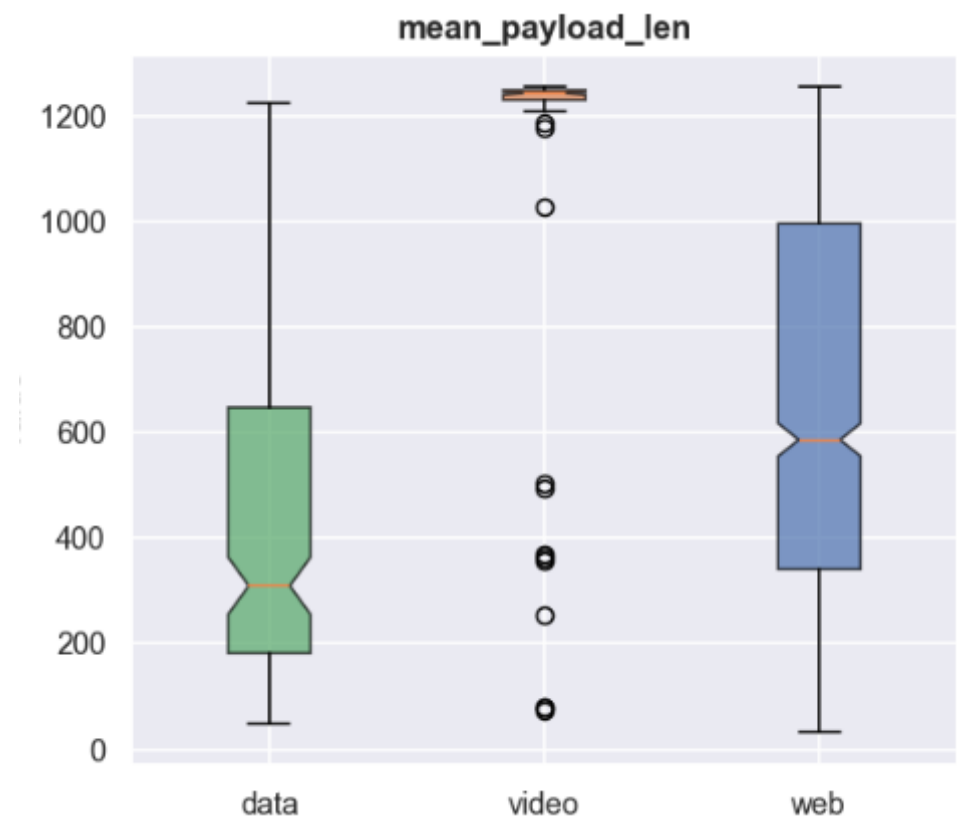
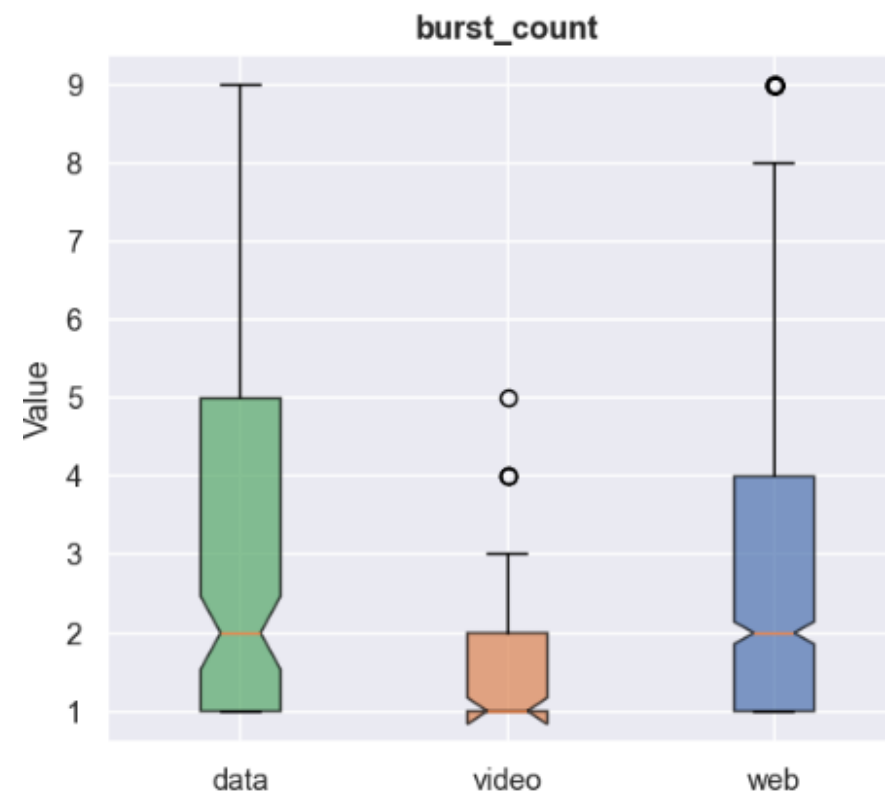
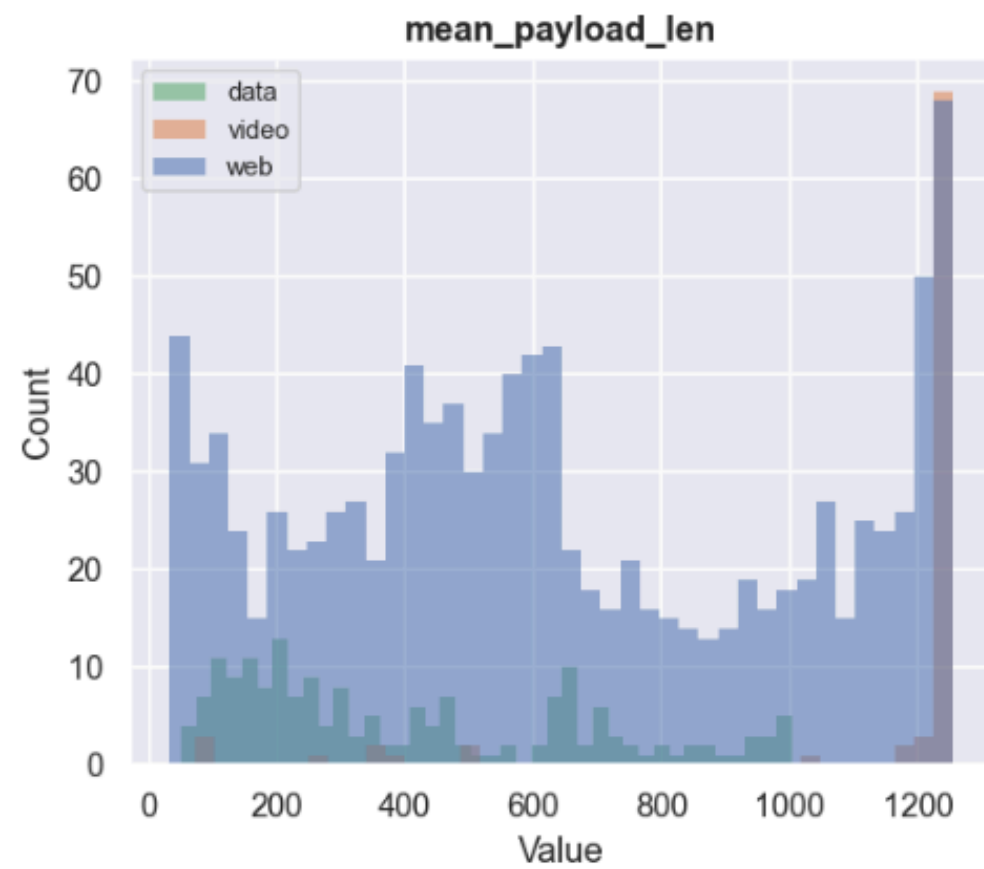


Feature Generation

Once flows are labeled, the raw traffic is sliced into **5-second time windows** (minimum 5 packets). We extract 14 distinct statistical features per window.

Category	Extracted Statistical Features
Volume	Packet Count, Total Bytes, Duration (s), Bytes/sec (Throughput)
Packet Size	Mean, Standard Deviation, Minimum, Maximum Payload Length
Inter-Arrival Time	Mean, Standard Deviation, Minimum, Maximum IAT
Behavior	Burst Count, Upload Ratio

Some Observations



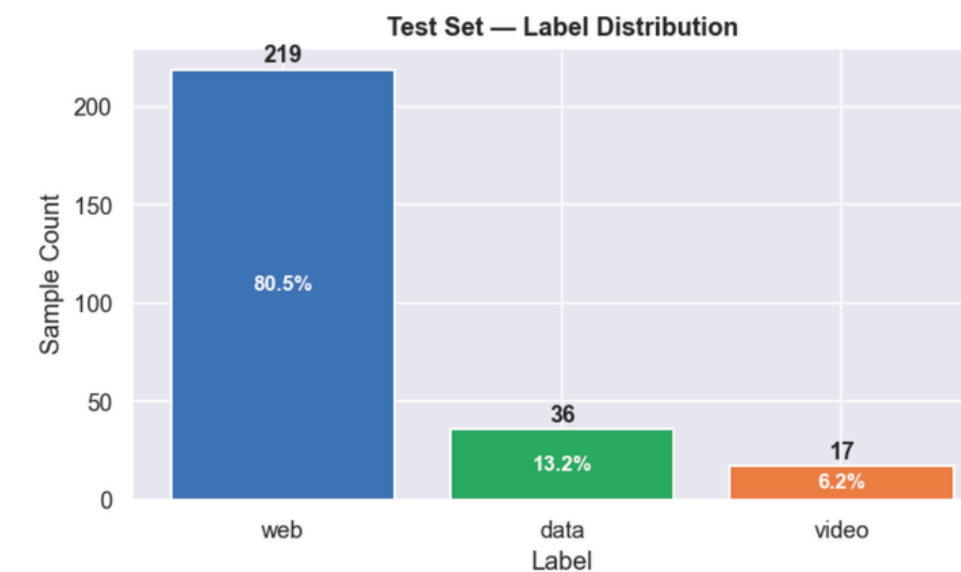
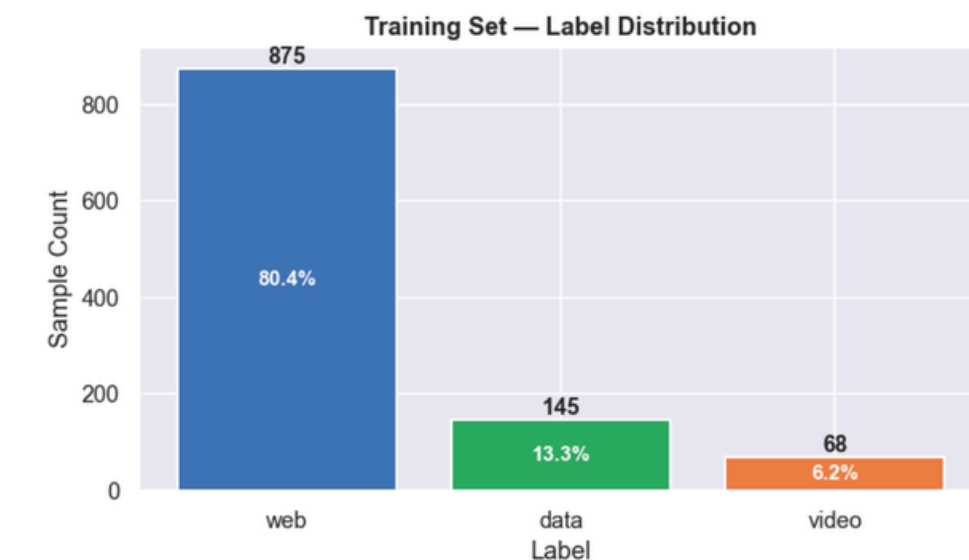
Addressing Data Imbalance

Exploratory Data Analysis revealed a massive skew in organic internet traffic, heavily favoring generic Web data.

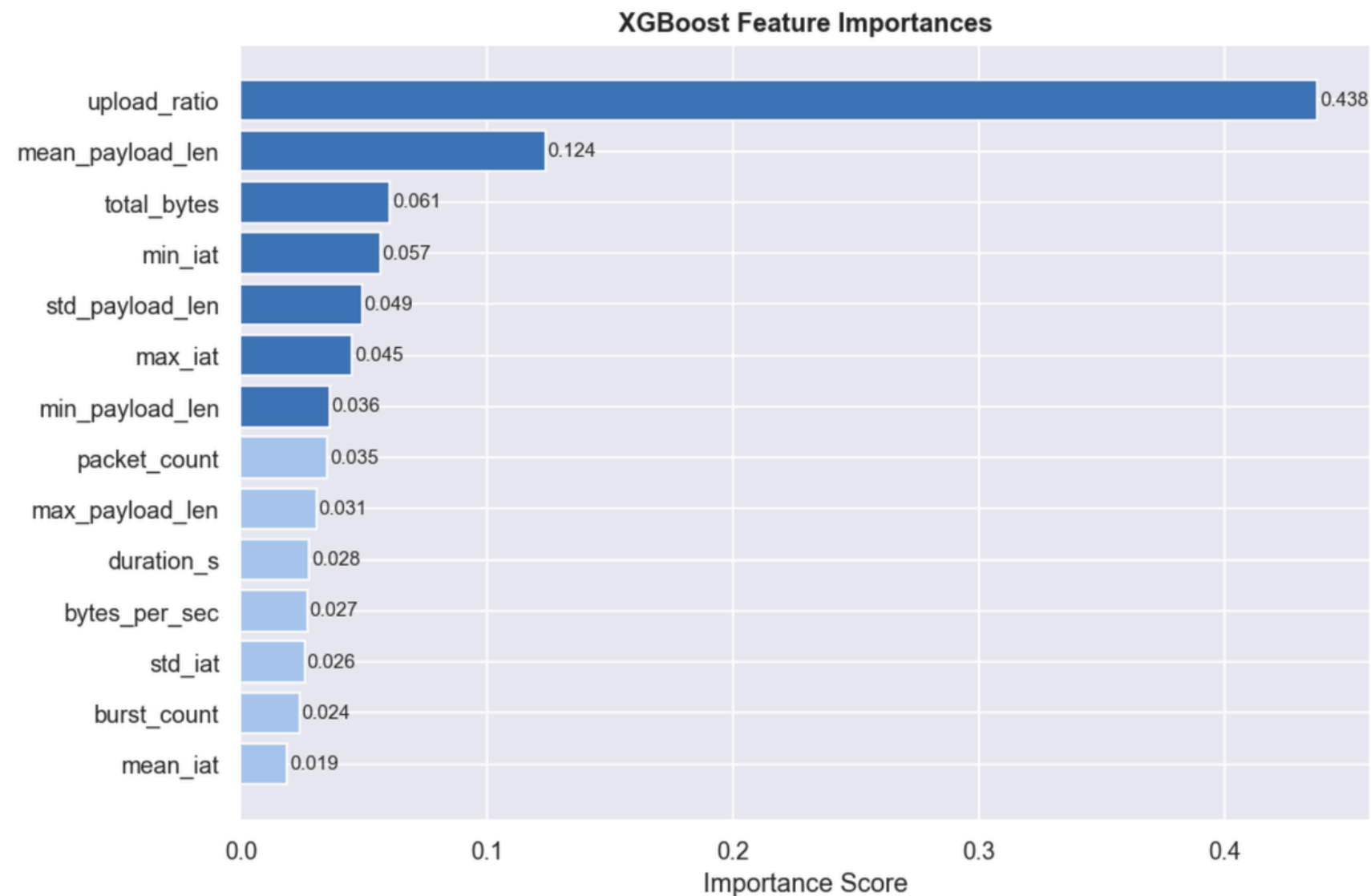
To prevent the XGBoost model from overwhelmingly biasing towards the majority class, we applied **SMOTE (Synthetic Minority Over-sampling Technique)**.

Class Label	Before SMOTE	After SMOTE
Web	875 samples	875 samples
Data	145 samples	875 samples
Video	68 samples	875 samples

EDA Observations



Training & Evaluation



88.6%

Average Cross-validation Accuracy

0.7886

Average Macro F1

Feature Importance Insights

The **Upload Ratio (0.438)** emerged as the single most critical predictor, successfully distinguishing heavily asymmetrical data downloads from symmetrical background web traffic, despite full payload encryption.

Model Parameters

- 300 estimators
- max_depth 6
- learning_rate 0.05.

Training & Evaluation

Model	Mean Accuracy	Std Accuracy	Mean Macro F1	Std Macro F1
XGBoost	0.886	0.011	0.789	0.019
Random Forest	0.875	0.011	0.777	0.02
Decision Tree	0.854	0.014	0.738	0.024

QoS Implementation Strategy

Hierarchical Token Bucket (HTB)

To enforce Quality of Service, we utilize HTB queues on the switch's egress port. HTB is designed to dynamically pace traffic rather than strictly blocking it.

Floors vs. Ceilings

HTB provides each class with a guaranteed minimum bandwidth (a floor) during heavy network congestion. However, if the network is idle, any queue can burst up to the full link capacity (a ceiling), ensuring no bandwidth is ever wasted.

Queue Roles & Class Weights

We established four distinct queues, each assigned a specific weight that dictates its share of the network pipe during contention:

Queue	Assigned Weight	Role
Queue 1: Video	4	High Priority Stream
Queue 2: Data	2	Medium Priority
Queue 3: Web	1	Low Priority
Queue 0: Default	1	Unclassified Catch-All

Traffic Shaping Workflow

1

Phase 1: Initial State

When the system boots, the four HTB queues are created on the egress port. Initially, only the **Default queue** is active. Because no flows have been classified yet, every incoming packet is routed to this default queue where it receives equal, first-come-first-served treatment with the full 10 Mbps available to it.

2

Phase 2: Flow Gets Classified & Steered

Once our ML model observes enough packets and predicts a flow's class (for example, Video), the switch dynamically registers that flow's 5-tuple. From that exact moment, the switch automatically steers all subsequent packets belonging to that specific flow directly into the Video queue instead of the Default queue.

3

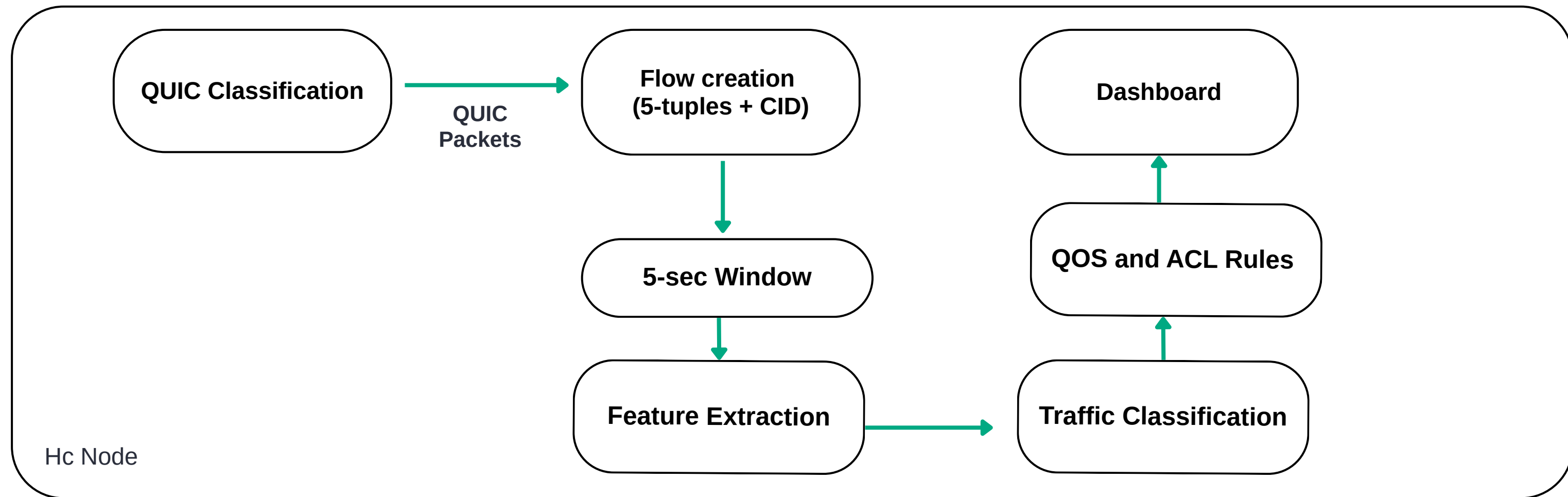
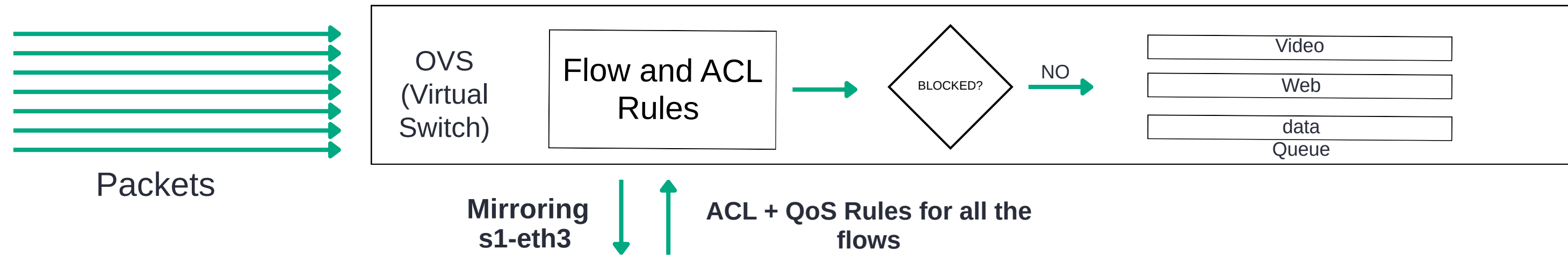
Phase 3: Bandwidth Rebalances Automatically

Whenever a new traffic class becomes active, the system automatically recalculates the guaranteed minimum bandwidth for every queue using a **weighted fair share**. For instance, if only Video (weight 4) and Data (weight 2) are active, Video is guaranteed 4/6ths of the bandwidth, and Data gets 2/6ths. The Default queue always retains whatever is left to prevent starvation. These updated rates are pushed to the kernel in real-time.

Demo Setup – Environment & Technology Stack

Components	Technology	Role
Network Emulation	Mininet 2.3.x + Open vSwitch 2.x	SDN topology with 3 hosts, 1 OVS switch, 10 Mbps links
Packet Capture	Linux AF_PACKET raw socket (Python 3.x)	Zero-copy capture on mirrored port (s1-eth3)
QUIC Detection	Custom PerPacketClassifier in python	Per-packet QUIC/non-QUIC classification using header analysis
Flow Aggregation	5-Tuple + Connection ID	Bidirectional flow mapping with QUIC connection migration support
Traffic Classification	XGBoost 3.2.0 + NumPy 2.4.6	14-feature vector per 5-second window – predicts Video / Data / Web
QoS Enforcement	OVS HTB queues + OpenFlow rules	4 queues (Video:4, Data:2, Web:1, Default:1 weight), dynamic bandwidth
ACL Enforcement	OpenFlow Drop Rules	Per-IP / per-category traffic blocking
SNI Extraction	QUIC Initial packet decryption (HKDF key derivation + AES-128-ECB header protection removal + AES-128-GCM payload decryption)	Extracts Server Name Indication from TLS ClientHello inside QUIC Initial packets (display only)
Dashboard	Flask 3.1.3 + Socket.IO 5.6.1 + vanilla JS	Real-time monitoring – flows, classification, QoS stats, ACL rules
Benchmarking	psutil 5.9+	CPU, memory, and timing profiling for edge feasibility

Simulation



Mininet

Resource Utilization Metrics

Test Conditions: Evaluated on a Mininet-emulated SDN topology with Open vSwitch (single switch, 10 Mbps HTB-shaped link) running on an Ubuntu host. Input: captured pcap containing 92,591 packets (83.5 MB) replayed at 4.45 Mbps / 589 pps via tcpreplay over 157 seconds. Full pipeline processed 74,554 QUIC packets across 129 flows, producing 128 classified windows. Resource metrics sampled over a 90-second window during active traffic replay.

Metrics	Definition	Measured Value	What It Indicates
CPU Utilization	Percentage of host CPU consumed by the complete pipeline — packet capture, QUIC/non-QUIC detection, 5-tuple + CID	4.8% mean, 26.6% peak	Low processing overhead on the test host. CPU per 1K PPS: 8.04% — the hardware-independent metric for estimating load on
Memory Footprint (RSS)	Resident Set Size — physical RAM occupied by the pipeline process, including Python runtime, loaded ML model, flow state, and	146.6 MB baseline, 169.1 MB peak	Peak consumption of 169 MB fits within a 512 MB edge device, leaving ~340 MB for OVS daemon, kernel buffers, and OS
Per-Packet Processing Time	Wall-clock time to process one QUIC packet through: Ethernet/IP/UDP header parsing, QUIC/non-QUIC classification, 5-tuple + CID-	24.8 us mean, 14.2 us median, 63.5 us P95	Processing capacity of ~40,000 PPS — 3x above the ~13,000 PPS required to sustain a 100 Mbps link at the observed 946-byte
ML Inference Time	Wall-clock time for one classification cycle: computation of 14 statistical features (packet size distribution, inter-arrival times,	4.38 ms mean, 2.97 ms median, 17.67 ms max	Classification completes in under 18 ms worst case — well within 100 ms budget; lightweight XGBoost model does not
Memory Growth Rate	Observed increase in RSS per new concurrent QUIC flow, including per-flow packet buffer, CID-to-flow mapping,	~320 KB per flow (observed, conservative estimate)	Estimated capacity of ~730-1,100 concurrent flows on a 512 MB edge device, depending on OS and OVS memory
OVS Rule Install Latency	Time from ML classification decision to OpenFlow enqueue rule being active in the OVS flow table. P95 reported to reflect	32.11 ms P95, 61.10 ms max	QoS enforcement activates within ~32 ms of classification; traffic experiences at most one unclassified default-queue interval
Model Storage Size	Total disk space required by ML model artifacts at deployment time	2.50 MB total (model.pkl: 1.81 MB, best_model.pkl: 0.69 MB)	Occupies <1% of a 256 MB edge device storage — no model compression or quantization needed

DEMO

Key Learnings



Raw Sockets Over Libraries

Re-implementing a purpose-built L2-L4 parser via raw Python sockets is vastly superior to heavy libraries for achieving zero-latency processing in constrained SDN namespaces.



Understanding QUIC Protocol Internals

Developed a comprehensive understanding of the QUIC protocol, including Long and Short Headers, Connection IDs, handshake packets, version negotiation, and how QUIC differs from traditional TCP/UDP communication.



ML for Real-World Network Traffic

Gained practical experience in feature engineering, model training (XGBoost), cross-validation, threshold optimization, SHAP-based interpretability, and evaluating models on unseen network traffic to ensure reliable deployment.

Future Scope

1

Broader Class Coverage

Extend beyond Video / Web / File to voice, gaming, IoT, and conferencing traffic categories.

2

Adaptive / Online Learning

Retrain the classifier on live traffic as QUIC implementations evolve such as new congestion controllers, encrypted CID rotation, QUIC v2.

3

Multi-Flow Correlation

Currently we classify each flow independently. Correlating flows per host could catch multiplexed applications more reliably.

4

Hardware Offload

Port the raw-packet path to eBPF/XDP or a SmartNIC for line-rate deployment on production traffic.

5

HPE Aruba Integration Study

Pilot the pipeline alongside an existing HPE Aruba data-plane stack to quantify real-world deployment cost, latency, and operational fit.

"The building block is proven. The next step is scale."

Conclusion

Major Project Outcomes

- Bypassed the QUIC encryption blindspot using raw byte heuristics and statistical flow modeling.
- Achieved an 88.6% real-time classification accuracy via an optimized, SMOTE-balanced XGBoost architecture.
- Proved the viability of translating ML predictions into autonomous, dynamic QoS policing via Open vSwitch.

Thank You

